Q-ORBIT — Four Optimization Approaches

Detailed implementation and evaluation report

Alphonsa · Angel · Rahul · Parikshit

Prepared 21 October 2026 | Current shared benchmark and hybrid v2

This report explains the four methods as they run in the common comparison framework. Original submissions and later adapters are distinguished throughout. All four methods run on classical computers; no quantum hardware is used.

The main validation supports lower average hybrid cost than the three primary baselines on the supplied suite at the recorded budget. It does not establish quantum advantage or a specific benefit from the QUBO stage. The no-QUBO hybrid was slightly cheaper on average.

Reading guide: Sections 1–2 define the common problem; Sections 3–6 explain the four approaches; Sections 7–8 detail the shared QUBO and evaluation; Sections 9–11 explain upgrades, limitations, reproduction, and evidence.

1. The routing problem and shared objective

CVRPTW requires assigning geographically dispersed customers to vehicles and choosing their visit order. Each customer must be served exactly once. Vehicles have finite capacities, customers have service durations and delivery windows, and every used vehicle must return to its depot within the depot deadline. The assignment and visit order interact: a short route may miss a time window, while a feasible detour can require an additional vehicle.

The shared benchmark extends this setting with departure-dependent directed traffic events, waiting through temporary closures, release times, and known new-order events. These inputs are known offline when search begins. This is planning with supplied traffic, rather than a validated live dispatch system.

Every compared solution uses the same monetary cost:

```
C = vehicle_rate × used_vehicles + distance_rate × distance
  + driver_rate × route_duration + fuel_price × fuel
  + lateness_rate × lateness
```

Distance is synthetic kilometres; duration is input minutes and includes driving, waiting and service; fuel is estimated litres. Fuel depends on load and congestion. Monetary coefficients are synthetic INR assumptions, not measured commercial delivery costs. Main experiments use hard windows, so feasible solutions have zero lateness. Soft-window runs are a different problem.

A route simulator propagates departure, travel, arrival, waiting, service and load in sequence. Changing an early visit can change all later visits. Structural violations such as missing or duplicate customers cannot receive valid quality scores. Full feasible operating cost is the final acceptance criterion.

The research question is whether a hybrid pipeline improves cost at a declared runtime budget compared with classical SA, tunnelling-inspired SA and QUBO+SB. CVRPTW is an established research problem with existing fleet-routing tools; this project studies a particular heuristic combination and its tradeoffs.

2. Common initialization and fair comparison

The framework builds initial routes using nearest feasible neighbours. If that leaves unassigned customers, up to eight feasible-insertion restarts are tried. Construction is a heuristic: failure does not prove that no feasible routing exists. A failure is recorded as NO_FEASIBLE_INITIAL_SOLUTION, and no finite partial-route cost is used in quality comparisons.

All methods receive the same complete initial routes for each case and seed. Construction is shared and timed separately. Solver time includes QUBO building, sampling, decoding, projection and dynamic candidate validation. Each method uses a bounded per-run complete-route cache keyed by vehicle and ordered customer list. An unchanged cached route avoids resimulation; changed routes are fully simulated. The cache is valid for an immutable problem/traffic snapshot.

Search deadlines are checked between atomic operations, so an action may finish after the requested limit. Actual elapsed times and overruns are recorded. Equal seeds pair experiments but do not imply identical random proposals across methods. Fixed-action runs support debugging; actions differ in computational effort.

Each method returns its best complete feasible solution, not merely its last visited state. A separate audit recomputes coverage, capacity, scheduling, traffic propagation and operating cost. Manifests and source snapshots identify the implementation actually used. Independently auditing a final route is not an independent proof that the search algorithm is optimal.

Code: team_benchmark/problem.py, cached.py, run.py, audit.py and verify_results.py. Settings and search dispatch: team_benchmark/solvers.py. Member entry points delegate to these components to keep the comparison contract consistent.

3. Alphonsa — Classical simulated annealing

Status and attribution. Alphonsa's folder originally had no submitted solver. With user authorization, a classical-SA baseline was implemented for her role; her review is pending. Entry point: members/Alphonsa/classical_sa.py. It uses shared search code and does not include a QUBO or tunnelling acceptance rule.

SA searches route lists directly. A random move relocates a customer, swaps two customers, or reverses a segment inside a route. Relocation may cross vehicles, including moving into an unused vehicle. Consequently SA can improve both vehicle assignment and route order. Capacity and scheduling are checked after the proposal rather than assumed to survive the move.

Let delta = candidate_cost − current_cost and T be temperature. Improving or equal-cost feasible moves are accepted. A worse feasible move is accepted with:

```
P(accept) = exp(−delta / T), when delta > 0
```

Accepting some worse moves helps leave local minima. Initially, temperature is max(1, 0.2 × initial_cost / customer_count). It cools by 0.995 per ordinary SA action, with a floor of 0.001 times initial temperature. After the configured stagnation threshold, the baseline returns to the best state and reheats. This avoids comparing the hybrid against an artificially frozen SA implementation.

```
current = best = shared_initial_routes
repeat until time/action limit:
    candidate = random relocation, swap or reversal
    compute complete feasible cost
    accept improvement, or accept worse move probabilistically
    retain new best; cool; reheat after stagnation
return best
```

Strengths: flexible customer reassignment, inexpensive proposals, simple implementation, and no binary-encoding overhead. Limitations: sensitivity to temperature and neighbourhood design, and difficulty coordinating several moves whose intermediate states violate hard windows. It provides no global optimum guarantee. Its role is a credible classical baseline rather than a weak control.

4. Angel — Tunnelling-inspired simulated annealing

Status and attribution. Angel's original members/Angel/saq_cvrptw.py is preserved unchanged. The shared entry point is members/Angel/run_tunnelling.py. Although an earlier role description called Angel's method QUBO+SA, her supplied algorithm is route-based tunnelling-inspired SA, and the user requested keeping that method.

It searches route lists without a QUBO. Its original proposal generator chooses relocation with probability 0.43, swap with probability 0.30, reversal with probability 0.18, and a larger multi-position perturbation with probability 0.09. Relocation and swaps can change customer assignment across vehicles. Large moves shuffle selected positions, bounded by the available customers and up to six selected positions in the implementation.

The main distinction is the acceptance law. For a worse feasible move:

```
P(accept) = exp(−d × sqrt(delta / T))
```

Here d is the proposal's width estimate. Relocation and swap return width 2; reversal uses its segment length with a minimum of 2; multi-position moves use the number of selected positions. This is a heuristic width, not an exact measurement of a physical barrier or a rigorous Hamming distance between all route encodings. Improving feasible moves are accepted normally.

For equal T, delta/T = 9 and d = 2, classical SA gives exp(−9), approximately 0.000123, while this rule gives exp(−6), approximately 0.00248. For delta/T = 1 and d = 2, it gives exp(−2), below classical exp(−1). Thus it does not simply accept more moves: both cost increase and width determine its behaviour.

The adapter reuses Angel's proposals and acceptance form, while applying the shared initialization, objective, temperature scaling, deadline and reheating. It bypasses the original standalone direct-trip precheck, raw mixed-unit cost and ID-index assumptions. Benchmark results therefore describe the adapted method, not an unchanged rerun of the standalone solver.

Strengths: global assignment changes and a different escape mechanism, with larger perturbations available. Limitations: heuristic width and parameter sensitivity; the preserved standalone issues remain documented. The word tunnelling describes a classical acceptance analogy, not quantum execution.

5. Rahul — QUBO plus simulated bifurcation

Status and attribution. Rahul's original ballistic SB implementation remains in members/Rahul/run_sb.py, with its tests and case files. The shared entry point is members/Rahul/run_comparison.py. The common comparison reuses his sampler with the common monetary objective and bounded route-window QUBO.

This comparator begins from the shared feasible routes and repeatedly selects bounded windows within existing routes. For k customers, k^2 binary variables represent customer-position assignments. A default five-customer window has 25 variables and 5! = 120 valid permutations. The solver searches a much larger binary space, so assignment penalties and decoding are necessary.

The QUBO $E(x) = x^TQx$ is converted using $x = (s + 1)/2$ to an Ising energy:

```
E(s) = constant + hᵀs + 0.5 sᵀKs
```

The implementation introduces an auxiliary spin to represent the linear fields as pairwise couplings. It normalizes the resulting coupling matrix by its spectral norm and simulates continuous position and momentum variables for multiple samples. One sample is warm-started near the current assignment.

```
for each integration step:
    pump increases from 0 to 1
    momentum += dt × [−(1 − pump) × position + coupling × J_force]
    position += dt × momentum
    clamp positions outside [−1, 1]; zero their momentum
recover binary scores using the auxiliary spin
```

This is symplectic-Euler ballistic simulated bifurcation with inelastic walls. Default backend integration uses dt = 0.2 and coupling = 1.0. Default shared sampling uses eight samples and 40 steps. Assignment-valid outputs are decoded; invalid outputs are projected to a one-to-one assignment using maximum-score matching. Unique decoded routes are checked with full dynamic evaluation, and only feasible cost improvements are retained.

Strengths: structured simultaneous route reordering and reuse of a tested quantum-inspired mechanical sampler. Limitations: fixed vehicle membership in this route-window comparator, quadratic variable growth, surrogate inaccuracies, and sampling/projection overhead. Local reordering alone cannot transfer customers to another vehicle or eliminate a route. SB runs classically and has no general optimality or quantum-advantage guarantee.

6. Parikshit — Adaptive hybrid v2

Status and attribution. members/Parikshit/run_hybrid.py runs the new shared hybrid. Parikshit's original qorbit package and tests are retained separately. The original package studied traffic-damage repair; the full global-SA/refinement/recombination cycle and temporary exclusion were added in the team framework, then upgraded.

The current pipeline is greedy construction → global SA → sustained stagnation → cross-vehicle repair → eligible QUBO refinement → full validation → further SA. Global SA supplies assignment flexibility. Classical repair supplies coordinated customer moves. The local QUBO supplies bounded alternative visit orders.

Refinement starts after 3 × plateau actions without a new best. With the default plateau of 50, the initial trigger is 150 stale actions. Successful refinement restores that interval; failure doubles it up to 24 × plateau. QUBO failures also postpone subsequent calls by increasing numbers of refinement cycles, capped at eight. Global SA continues between calls, avoiding repeated expensive failed sampling at every short plateau.

Cross-vehicle repair removes customers around avoidable detours or attempts to empty a short route to save fixed vehicle cost. Regret-2 insertion prioritizes customers whose second-best feasible insertion is substantially worse than their best. Positions are shortlisted geometrically, then ranked by exact feasible marginal operating cost. Search examines up to three positions in each of four candidate vehicles and at most 160 exact insertions per repair attempt. Deadline interruption or incomplete reinsertion discards the entire candidate.

Eligible QUBO windows are ranked by avoidable detours rather than simply the largest total route cost. Attempt history limits repetition; selected customers are temporarily excluded for two subsequent refinement cycles by default. Repair selections also enter the exclusion history. Exact dynamic feasibility and true operating cost determine QUBO acceptance, then global SA resumes.

The local QUBO backend defaults to classical binary SA. The optional --hybrid-backend sb selects Rahul's ballistic SB. QUBO specifies the binary model; it does not require a quantum sampler. Classical binary SA here searches assignment bits, while the outer global SA searches complete route lists.

Strengths: combines fleet reassignment, coordinated repair and local sequencing; adapts effort to stagnation. Limitations: more components and parameters, bounded heuristic insertion search, and no proven independent QUBO benefit in current data.

7. Shared QUBO: what it represents

For a selected window, x(i,p) = 1 means customer i occupies position p. The model includes cost of entering the window, exiting it, adjacent visits, and approximate waiting/lateness. Assignment constraints require each customer once and one customer at every position:

```
E(x) = surrogate_travel_and_timing(x)
     + A × sum_i (sum_p x(i,p) − 1)²
     + A × sum_p (sum_i x(i,p) − 1)²
```

The implementation expands this into a symmetric matrix, handling the constant offset separately. The assignment penalty A exceeds a bound on the entire maximum valid surrogate cost. This is stronger than assuming that normalizing individual coefficients alone ensures assignment feasibility.

That penalty statement concerns assignments in the surrogate. It does not guarantee that a returned sample minimizes the QUBO, or that its decoded route satisfies hard delivery windows. Sampling is heuristic, and timing/load effects are frozen or approximated while coefficients are built. An earlier reordered customer changes downstream arrival times, so full-route recomputation remains necessary. Capacity, coverage and scheduling are not all exactly encoded inside this small QUBO.

As an example, moving one customer earlier may reduce geometric distance while causing waiting, congestion at a different departure time, or a missed downstream window. A lower surrogate energy may therefore fail to improve the true cost. Recorded predicted-versus-actual improvement, raw-valid sample counts, projected assignments and accepted-candidate counts help diagnose those mismatches.

Projection repairs the one-to-one assignment structure using score matching. It does not solve full CVRPTW feasibility, and a projected solution's quality cannot automatically be credited to an optimal sampler output. Full acceptance checks and no-QUBO ablations are essential to attribution.

Source locations: team_benchmark/solvers.py::build_qubo and the decode/project, solve_sa, solve_sb and qubo_to_ising functions in members/Rahul/run_sb.py.

8. Main validation: measured comparison

The frozen v2 protocol used seeds 10–14 and 0.5 seconds per optimization. TC03, TC08 and TC11 had been used for development and were excluded. TC14 contains CSV despite its JSON extension and was recorded as invalid. Of the remaining 25 valid files, TC15, TC16 and TC25 failed common construction. Across seven methods there were 875 attempts: 770 feasible results and 105 construction failures. All 770 feasible results passed independent audits.

Positive percentages below mean lower full-hybrid cost. Paired differences are averaged over seeds within cases, then across 22 feasible cases. Intervals are case-bootstrap 95% intervals; tests are two-sided Wilcoxon with Holm correction.

Comparator	Reduction	95% interval	Holm p
Classical SA	1.50%	0.84 to 2.22	0.0009822
Angel tunnelling SA	1.11%	0.60 to 1.69	0.0009822
QUBO+SB	8.12%	5.98 to 10.30	0.0005314
Equally cached v1 hybrid	3.16%	1.96 to 4.42	0.0009822
V2 without repair	1.66%	0.88 to 2.56	0.0009822
V2 without QUBO	−0.41%	−1.04 to 0.12	0.3627

The full pipeline improves mean cost over the primary three baselines on this suite and budget. Classical repair accepted 196 improvements in 796 attempts. QUBO calls fell from 1,857 in equally cached v1 to 278 in v2, with only three accepted QUBO improvements after repair. The no-QUBO variant was cheaper on average, and its paired interval crossed zero. These findings support the pipeline improvement without establishing a QUBO-specific gain.

The maximum v2 overrun was 17.7 ms; equally cached v1 reached 34.6 ms. Related supplied fixtures are not independent real-world holdout evidence, even with new seeds. The comparison measures the implemented methods, including the route-only scope of QUBO+SB, rather than every possible algorithm in those families.

Evidence: results/comparison/hybrid_v2_validation/REPORT.md, protocol.json, runs.json, runs.csv, paired_comparisons.json, manifest.json and audit.json.

9. Earlier results and upgrades

V1 established the shared comparison and original hybrid cycle. Its supplied suite had 1,120 attempts and 1,000 audited feasible results. Hybrid mean cost reductions were −0.25% versus classical SA, −0.89% versus Angel and +6.13% versus QUBO+SB. It therefore did not outperform both global-SA baselines. Additional Solomon checks yielded 100 audited feasible results from 120 attempts, with mixed findings. These historical outcomes remain in v1-comparisons.zip.

V2 addressed overhead and search limitations: complete-route caching for every method; bounded cross-vehicle regret repair; adaptive stagnation and failure backoff; detour-based window ranking; both local backends; and work diagnostics. An equally cached v1 reference separates search changes from caching benefits. No-repair and no-QUBO ablations test the components. Development used TC03, TC08 and TC11, seeds 0–2, at 0.25 seconds; those observations are archived separately and do not count as final validation.

A separate frozen-code 1-second check used TC06, TC17, TC18 and TC20 with five seeds and six methods: all 120 results were feasible and audited. Hybrid mean reductions were 1.07% versus classical SA, 0.81% versus Angel, 11.19% versus QUBO+SB, and 1.71% versus equally cached v1. The no-QUBO variant was cheaper by 0.67% on average. With only four cases, Holm-adjusted tests did not establish superiority. These results must not be pooled with the 0.5-second observations as independent samples or used as a clean general budget-scaling experiment.

The later repository cleanup removed placeholders and duplicate scaffolding, centralized tests, added a working read-only results viewer, and compressed historical evidence after hash verification. It did not change recorded solver results. The latest recorded test run passed 134 substantive tests. The 770 main plus 120 budget-check feasible results were independently re-audited.

Detailed upgrade chronology: docs/PROJECT_MASTER_RECORD.md. Historical findings: docs/history/TEAM_COMPARISON_SUMMARY.md. Current changes: docs/HYBRID_V2.md.

10. Practical differences and remaining work

Classical SA is the simplest fleet-wide reference. Angel's method also searches fleet-wide but changes the move mixture and worse-move acceptance law. QUBO+SB reorders bounded windows with fixed customer membership. Hybrid v2 combines global SA, coordinated classical repair and adaptive local QUBO calls. Their neighbourhoods and overhead differ; equal time does not make their internal work or scope identical.

No approach guarantees the globally best large-instance solution. Tiny exact checks establish correctness and optimality only on those tiny instances. Synthetic traffic, model-estimated fuel and operating coefficients limit claims about real savings. The pipeline does not implement a validated live event simulator, executed-prefix freezing, online new-order insertion or breakdown rescue. A fixed cache cannot remain valid through arbitrary live traffic changes.

The highest-priority research question is whether a revised QUBO stage adds enough value to justify its overhead. Investigate surrogate errors, assignment projection and sampler quality against equally budgeted direct permutation or classical window search. Keep a no-QUBO control and freeze proposed settings before evaluating new independent data. Current outcomes must not be retuned and reported as if they came from an untouched holdout.

Other next steps are improving common construction, obtaining Alphonsa's review, adding a mature external VRP baseline, testing independently held-out instances and longer budgets, and validating cost assumptions on operational data. Apply construction and evaluator improvements equally across compared methods.

A defensible presentation statement is: “On the stated supplied validation suite and time budget, adaptive hybrid v2 reduced mean cost relative to the three implemented baselines. Cross-vehicle repair contributed useful gains; the current experiment did not establish an independent QUBO benefit.”

11. Reproduction and source guide

Run commands from the repository root in PowerShell. Existing folders with recorded runs are protected; choose a new output name for every new experiment. The examples below use the existing environment. Fresh environments install requirements-dev.txt for runtime and testing dependencies.

```
# Inspect settings and available method names
venv\Scripts\python.exe -B -m team_benchmark.run --help

# Run one hybrid case; default local backend is SA
venv\Scripts\python.exe -B members/Parikshit/run_hybrid.py
    members/Rahul/test_cases/TC11.json --seconds 1

# Add --hybrid-backend sb to use ballistic SB locally

# Tests
venv\Scripts\python.exe -B -m pytest -q

# Saved results audit
venv\Scripts\python.exe -B -m team_benchmark.verify_results
    results/comparison/hybrid_v2_validation

# Recorded-results dashboard
venv\Scripts\python.exe -B -m streamlit run dashboard/app.py
```

Wrapped command lines above are for page readability; enter each command on one line. Complete runnable comparison commands appear in docs/experiments.md.

Primary implementation references are team_benchmark/problem.py, solvers.py, hybrid.py and cached.py; members/Angel/saq_cvrptw.py; and members/Rahul/run_sb.py. The four role entry points are listed in members/README.md. Original member packages should not be confused with the common-evaluator adapters.

Current evidence folders are results/comparison/hybrid_v2_validation and results/comparison/hybrid_v2_budget_check. Each preserves raw runs, protocol, manifest, audits and exact source bytes in sources.zip. Historical evidence is in results/archive/, with per-file and archive integrity records. The PDF is a readable synthesis; source code and recorded experiment artifacts remain the underlying evidence.

Rebuild this PDF using docs/tools/build_approaches_pdf.py. Its editable source is docs/FOUR_APPROACHES.md. This report's content is based on local code and saved evidence; no new scientific benchmark was executed to produce it.